

Faculdade de Tecnologia Professor Francisco de Moura – FATEC Jacareí

Equipe Undefined

Relatório da Sprint 1

Todos os itens da rubrica foram avaliados; este documento lista somente itens em que foram encontradas falhas ou inconformidades.

Quando um item não aparece a seguir, não foi registrada falha objetiva para ele na inspeção realizada.

Engenharia de Software II

Item avaliado	Falha encontrada
1.1 README principal	O README principal descreve requisitos, tecnologias e backlog, mas não apresenta estrutura de pastas nem instruções de execução suficientes para reproduzir o ambiente. Também referencia a imagem ./docs/sprint1/burndown.png, que não existe no repositório inspecionado.
1.2 README das pastas principais	Não há README.md na pasta backend. O README do frontend permanece como texto padrão do template React + Vite, sem explicar a responsabilidade da pasta no projeto. A documentação específica encontrada concentra-se em backend/db e backend/db/init.
2.1 Backlog do Produto	O Product Backlog aparece como lista de requisitos e user stories, mas não contém prioridades explícitas. Assim, não atende integralmente ao critério de backlog com funcionalidades, prioridades, descrição das histórias e critérios de aceite.
2.2 Backlog da Sprint 1	O backlog da Sprint 1 no README não informa responsáveis por tarefa. O arquivo docs/backlog/backlog_sprint1.csv contém apenas dados de burndown (Data, Real, Ideal), não um backlog com tarefas, responsáveis, status e vínculo funcional.
2.3 Definition of Done (DoD)	Há DoD por user story, mas não foi encontrado um DoD geral do projeto/sprint. Além disso, itens marcados como concluídos na Sprint 1 não cumprem o próprio critério declarado: por exemplo, “Criar service de API no frontend” está concluído no README, mas o frontend faz fetch diretamente em ChatContainer.tsx, sem camada de service.
3.1 Diagrama de Casos de Uso	O diagrama mistura “Usuário” e “Aluno” com generalização pouco clara, usa casos de uso com verbos de implementação como “Armazenar FAQs” e “Armazenar status da pergunta”, e deixa

Faculdade de Tecnologia Professor Francisco de Moura – FATEC Jacareí

	relacionamentos importantes incompletos, como registro de log apenas para algumas ações. Isso reduz a correção conceitual do diagrama.
3.2 Coerência entre documentação e implementação	A documentação declara Sprint 1 concluída para service de API, chamada de endpoint, atualização de estado por resposta da API, árvore integrada e burndown, mas a implementação mantém acesso à API dentro do componente ChatContainer e o burndown referenciado não está versionado. Há também divergência entre RF02, que prevê documentos/chunks/metadados estruturados, e o schema atual, que remove documents e document_chunks.
4.1 Estratégia de branches	Foram encontradas apenas main e origin/develop, sem branches de feature/fix locais ou remotas na inspeção. A organização mínima de branches de trabalho não ficou evidenciada.
4.2 Distribuição temporal dos commits	A distribuição de commits não cobre a sprint de forma uniforme: há 10 commits em 28/04/2026 e 5 commits em 05/05/2026, data da sprint review, com outros dias tendo volume bem menor.
4.3 Participação dos membros	A participação por commits está desequilibrada: Pedro augusto tem 14 commits, João Sai Hack 8, Pedrinnnnn 5, MarcusVRDN 3 e Undefined 2. A documentação lista mais integrantes do que os autores observados no shortlog local.

Desenvolvimento Web II

Item avaliado	Falha encontrada
3.1 Uso de variáveis de ambiente	O docker-compose.yml usa variáveis, mas define defaults inseguros para JWT_SECRET e credenciais de banco/pgAdmin.
3.2 Exposição de dados sensíveis	O seed cria usuários com senhas conhecidas hardcoded (Admin@123 e Secretaria@123). Mesmo com hash no banco, as senhas originais ficam versionadas no repositório.
4.2 Organização lógica do frontend	Não há camada frontend de services, hooks ou contexts para acesso à API. As chamadas fetch, montagem de URLs, controle de histórico e regras de navegação estão concentradas em ChatContainer.tsx, dificultando manutenção e testes.
4.3 Estrutura visual	O componente MessageBubble usa estilos inline extensos para o bloco de evidência, reduzindo reutilização do CSS. Também há console.log em getFileName, vazando ruído técnico em execução.

Faculdade de Tecnologia Professor Francisco de Moura – FATEC Jacareí

Segurança frontend	MessageBubble renderiza answer_summary vindo da base com dangerouslySetInnerHTML sem sanitização. Como o conteúdo do chatbot é administrável pelo banco, isso abre risco de XSS quando houver gestão de conteúdo.
--------------------	---

Banco de Dados Relacional

Item avaliado	Falha encontrada
1.2 Persistência correta	A árvore principal usa navigation_nodes, mas o repositório de conhecimento exigido não está persistido de forma completa: documentos oficiais, chunks e metadados estruturados não existem como tabelas próprias; trechos e fontes ficam embutidos no nó de navegação.
2.1 Qualidade dos comandos SQL	O schema cria e depois remove documents, document_chunks e faq_entries, contrariando o RF02 e o README, que declaram repositório com documentos oficiais, chunks e metadados. Isso indica incoerência de modelagem para o domínio do desafio.
2.1 Qualidade dos comandos SQL	O seed possui slugs duplicados: dsm-portfolio, geo-portfolio e marh-portfolio aparecem duas vezes. Como o upsert usa ON CONFLICT(slug), a segunda entrada sobrescreve a primeira, eliminando opções distintas como Portfólio e TG/TCC.
2.1 Qualidade dos comandos SQL	O seed referencia assets de horários que não existem em frontend/public/assets/png, como horario-aula-1geo.png, horario-aula-2geo.png, horario-aula-1marh.png e outros. Esses nós retornam links quebrados ao usuário.
2.2 Otimização inicial	Há índices para parent_id, status e session_id, mas não para slug, apesar de slug ser o principal filtro usado pelo backend em findChildrenBySlug. A unicidade cria índice, mas a documentação não explicita essa dependência, e não há índice composto para parent_id/is_active/display_order, consulta usada na navegação.

Técnicas de Programação I

Item avaliado	Falha encontrada
1.1 Aplicação funcional	Não foi possível comprovar build local da aplicação. O frontend falhou com ausência de tipos vite/client e node, e o backend falhou com ausência de módulos/tipos express, pg, cors e node. O backend não possui node_modules instalado no workspace;

Faculdade de Tecnologia Professor Francisco de Moura – FATEC Jacareí

	portanto a execução local direta não ficou reproduzível sem etapa adicional.
1.2 Execução containerizada	O Docker Compose valida a configuração e inclui PostgreSQL, backend e frontend, mas o backend/Dockerfile não copia o código-fonte para a imagem. Ele depende do bind mount do Compose para existir /app/src, então a imagem backend isolada não é executável de forma autônoma.
2.1 Tipagem TypeScript	Há tipagem explícita nos componentes principais, mas o build TypeScript não pôde ser validado no estado local do repositório. No backend, os erros incluem parâmetros req/res implicitamente any quando os tipos de Express não estão resolvidos.
2.2 Uso de conceitos de POO	Não há uso relevante de classes, encapsulamento ou abstrações orientadas a objeto. A implementação é majoritariamente procedural/funcional em controllers, services como objetos literais e componentes React; isso atende parcialmente ao stack, mas é fraco para o critério específico de POO.